

Cipher Brain



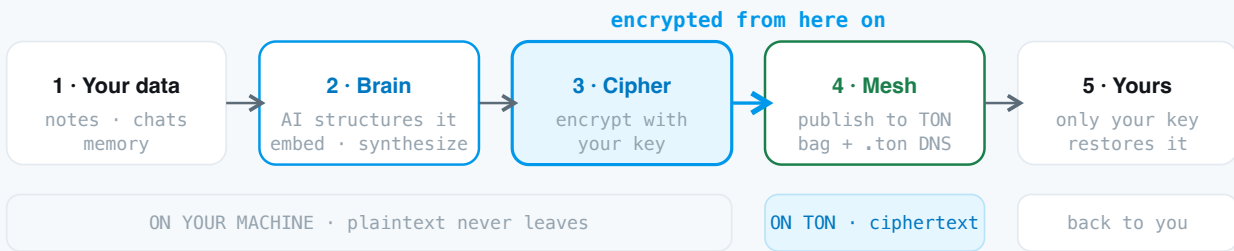
The brain you offload to AI
**should be unreadable, and
impossible to delete.**

An AI second brain, **encrypted on your machine** —
only you hold the key — stored where no platform can
read or delete it.

01 CONCEPT

Encrypt your memory locally, store it where no one can reach it

Encrypt the notes and chats you feed to AI **on your own machine**, turn
them into a queryable **second brain**, and publish it to **TON Storage + .ton**
DNS. Three layers: **Cipher, Brain, Mesh.**



Steps 1–3 run on your machine – only encrypted data ever reaches TON.

02 PROBLEM

Your brain now lives on someone else's server

The more you offload to AI, the more your **second brain lives inside a platform** — deletable, alterable, and readable by whoever runs it. Your most personal data is the one you own least.

BRAIN INSIDE A PLATFORM



- ✗ deleted on a policy change
- ✗ readable by the operator
- ✗ censorable / subpoenaable

CIPHER BRAIN



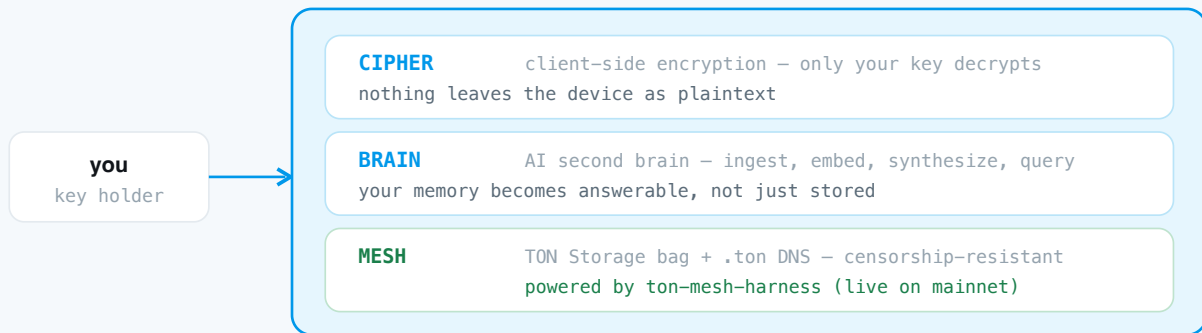
- ✓ no operator can read it
- ✓ no host to delete it
- ✓ updates need your signature

Same second brain, two custody models – only the encrypted-on-TON one is truly yours.

03 SOLUTION

Encrypt it. Own the key. Put it where it can't be taken down.

Three layers in one tool: **Cipher** encrypts on-device, **Brain** makes your memory queryable, and **Mesh** (the shipping [ton-mesh-harness](#)) puts it beyond any single party's reach.



Cipher + Brain run on your machine; Mesh carries the encrypted result onto TON.

Cipher

Client-side encryption
keyed to you — the
network only ever holds
ciphertext.

Brain

An AI second brain over
your own corpus —
memory you can *ask*,
not just store.

Mesh

TON Storage + .ton
DNS via ton-mesh-
harness — no host to
seize, owner-signed
only.

04 WHY TON

Censorship resistance needs storage *and* naming — TON has both

An encrypted blob is useless if its address can be seized. TON is the one stack that pairs **immutable storage** with an **on-chain name no one can take** and a **private transport** — built by Telegram's team as a digital-resistance stack.

TON Storage

Content-addressed
bags — immutable, no
host to subpoena.

.ton DNS

A human-readable name
updated only by your
signature — unseizable.
IPFS / Arweave have no
equivalent.

TON Proxy

Garlic-routed access
hides who reads what —
privacy at the transport
layer.

05 STATUS & ROADMAP

Two halves already run — Cipher Brain bridges them

Not a slide. The **Mesh** and **Brain** layers each run today; the **Cipher** layer and the wiring between them are what's left.

STEP 1 Mesh layer — censorship-resistant publishing DONE · mainnet

`ton-mesh-harness` is open-source on npm with verified on-chain deploys on TON mainnet. One command ships any build to a bag + `.ton` DNS — and an AI agent can drive it via MCP.

```
# the Mesh layer, available now
$ npx ton-mesh-harness ./build --domain mybrain.ton --site-auto
```

STEP 2 Brain layer — a second brain we already run RUNNING · GBrain

We already run a personal second brain (**GBrain**) nightly on a local machine: it **ingests our AI conversations across projects, embeds them, and synthesizes links**, then serves hybrid search + LLM reasoning over the whole corpus. The Brain layer is real, not a mock.

```
# the brain we run every night - locally
$ gbrain think "what did we decide about X?"
```

STEP 3 Cipher — encrypt & bridge the two ROADMAP

Encrypt the brain client-side and publish it through the Mesh layer, into a bag **only your key can restore**. Both halves exist — what's left is the Cipher layer and the wiring between them.

```
// the shape we're building toward
$ cipher-brain push --key ~/.cipher-brain/key --domain mybrain.ton
```

Where the prototype ends: the Mesh layer (ton-mesh-harness) ships on mainnet and the Brain layer (GBrain) runs nightly — but they aren't joined yet, and the **Cipher** layer isn't built. We say it plainly because **censorship-resistant is not the same as private** — closing that gap is the point of Cipher Brain.

ton-mesh-harness (Mesh layer) TON DNS TON Connect